

Reviewer Pack — Tropical Cyclotomic Polynomial Rank Bounds for Boolean Circuits

Entry d68e535d9333 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 03:20:02 UTC

Tropical Cyclotomic Polynomial Rank Bounds for Boolean Circuit Size

Entry ID: d68e535d9333 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 03:20:02 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Cyclotomic Polynomials) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

{‘sentence_1’: ‘For a given number of variables n , the rank of the tropical cyclotomic polynomial associated with a Boolean circuit C is at most polynomial in the size of C .’, ‘sentence_2’: ‘Equivalently, for a Boolean circuit C computing a function f , the smallest integer r such that f can be represented as a tropical cyclotomic polynomial over the finite field with 2^r elements is bounded by the complexity class of C .’, ‘sentence_3’: ‘This bound holds for all circuits C of size at most 40.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Tropical cyclotomic polynomials provide a unique way to represent functions in a tropical setting, which may reveal new structural properties not evident in classical settings.’, ‘sentence_2’: ‘The conjecture bridges tropical geometry with boolean circuit complexity, potentially leading to new insights into the inherent structure of Boolean functions.’, ‘sentence_3’: ‘If true, it would offer a constructive method for bounding the size of circuits using tropical algebraic tools.’}

Taxonomy category: TROPICAL_CYCLOTOMIC_POLYNOMIAL_RANK (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f48f73bfd96df46c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all circuits C with at most 40 variables, the rank of the associated tropical cyclotomic polynomial is less than or equal to the 5th root of the circuit’s size (in terms of gates), and supported by at least 95% of the seeds. It is falsified if any seed produces a tropical cyclotomic polynomial rank exceeding the 5th root of the circuit’s size.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry cyclotomic polynomial rank AND boolean circuit complexity - Boolean circuit size tropical cyclotomic polynomial rank bound - finite field with 2^r elements Boolean circuit complexity representation

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1207.1925v1>] Introduction to tropical algebraic geometry - [<http://arxiv.org/abs/1206.1925v1>] Counting Algebraic Curves with Tropical Geometry - [s2:1710.09502] Barriers for Rank Methods in Arithmetic Complexity - [s2:10.1145/3651588] Tight Bounds of Circuits for Sum-Product Queries

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_boolean_circuit(n):
    if n == 1:
        return ['x1']
    left = generate_boolean_circuit(n // 2)
    right = generate_boolean_circuit(n - n // 2)
    return [f'({l} AND {r})' for l in left] + [f'({l} OR {r})' for l in left] + [f'(NOT {l})' for l in left]

def evaluate_circuit(circuit, assignment):
    stack = []
    for token in circuit:
        if 'x' in token:
            stack.append(assignment[token[1:]])
        elif token == 'NOT':
            stack.append(not stack.pop())
        else:
            b2 = stack.pop()
            b1 = stack.pop()
            if token == 'AND':
                stack.append(b1 and b2)
            elif token == 'OR':
                stack.append(b1 or b2)
```

```

    return stack[0]

def generate_random_assignment(n):
    return {f'x{i+1}': random.choice([True, False]) for i in range(n)}

def tropical_cyclotomic_polynomial(circuit):
    n = len(circuit)
    assignment = generate_random_assignment(n)
    value = evaluate_circuit(circuit, assignment)
    rank = 0
    while value != 0:
        rank += 1
        value = (value + 1) // 2
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        circuit = generate_boolean_circuit(n)
        rank = tropical_cyclotomic_polynomial(circuit)
        bound = math.pow(len(circuit), 1/5)
        results.append({
            "n": n,
            "rank": rank,
            "bound": bound
        })
    metric_value = sum(result["rank"] for result in results) / len(results)
    conjecture_holds = all(result["rank"] <= result["bound"] for result in results)
    counterexample = "" if conjecture_holds else f"n={results[0]['n']}, rank={results[0]['rank']}"
    return {
        "metric_name": "Rank of Tropical Cyclotomic Polynomial",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30*3 + 1))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = results[results.index(next(r for r in results if not r["conjecture_holds"]))]

```

```
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39a75c70.py", line 82, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39a75c70.py", line 58, in run_trial
    circuit = generate_boolean_circuit(n)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39a75c70.py", line 21, in generate_bool
    left = generate_boolean_circuit(n // 2)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39a75c70.py", line 23, in generate_bool
    return [f'({l} AND {r})' for l in left] + [f'({l} OR {r})' for l in left] + [f'(NOT {l})' for l in right]
           ^
```

NameError: name 'r' is not defined. Did you mean: 're'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11605
2	preregistration	ollama_remote	glm4:latest	0	0	5940
3	novelty	ollama_remote	glm4:latest	0	0	4588
4	novelty	ollama_remote	glm4:latest	0	0	6494
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12915
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10354
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12807
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10191
9	judge	ollama_remote	glm4:latest	0	0	8375

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 83268 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d68e535d9333.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d68e535d9333.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d68e535d9333.tar.gz (if generated)